Lesson 13: A Hit That Reacts

Godot Engine

VIDEO GAME CREATION

This week: getting hit finally LOOKS like getting hit. A brief flash, then a small explosion, before the enemy disappears.

Open your own [YourName]Game project from Lesson 12 - don't start a new one.

Important: your score, lives, wave, and win/lose logic should all still work exactly as before. Today only changes how the enemy's removal LOOKS, not when your game-state actually changes.

Setup

1. Open your [YourName]Game project from Lesson 12.
2. Create a new scene with a Sprite2D as its ROOT node (not a Node2D/other wrapper with a Sprite2D nested underneath it - the Sprite2D itself must be the root). Name it Explosion, set its Texture to explosion_placeholder.png.
3. Add a Timer node as a DIRECT CHILD of that Sprite2D root. In the Inspector, set Wait Time = 0.2, turn One Shot on, AND turn Autostart on - without Autostart, the Timer never begins counting down on its own, and the explosion will sit on screen forever.
4. Attach a brand-new script to the Sprite2D root node - this is a separate script from enemy.gd; the explosion manages its own cleanup with zero help from the Enemy or main scene. In its _ready(), connect the Timer's timeout signal to a function that calls queue_free() (code below). Save the scene as explosion.tscn.
5. Import enemy_invader_white.png (this lesson's Assets folder) into your project - it's the same enemy silhouette as Lesson 4's sprite, pre-recolored solid white, used below for the hit flash.

Level 1 - Minimal (MVP)

On every hit, the enemy briefly flashes, and a small independent explosion effect appears and cleans itself up. Score, lives, wave, and win/lose logic are all unaffected.



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.

6. On the Enemy scene's script, find _on_hitbox_area_entered() - by now (since Lesson 12 made Lesson 4's Level 3 sound mandatory) it already destroys the bullet, disables the Enemy's own Hitbox, plays the hit sound, hides the Enemy immediately, and awaits the sound's own finished signal before queue_free(). This lesson removes hide() entirely: the Enemy needs to stay VISIBLE now, so the new flash can actually be seen.
7. Right after disabling the Hitbox and playing the hit sound, instantiate explosion.tscn at global_position and add it to the scene (code below) - this happens immediately and independently, right alongside the existing sound, with zero effect on anything else. The explosion scene's own script (from Setup) is what makes it clean itself up a moment later - enemy.gd only ever spawns it and never touches it again.
8. Swap the Enemy's texture to enemy_invader_white.png for the flash, then keep the existing destroyed.emit() call running immediately after that, unchanged - score/wave/win-check must happen at exactly the same instant as last week. Only THEN await get_tree().create_timer(0.1).timeout to pause this function for a tenth of a second, with queue_free() moved to after the await. A color tint (modulate) won't work here: modulate multiplies the sprite's existing color, and multiplying any color by white leaves it completely unchanged - a genuinely white sprite is the only way to get a visible white flash on colored source art, the same reason Lesson 2's Recolor swaps sprites instead of tinting them.
9. The old await $Hit.finished is replaced by this new await get_tree().create_timer(0.1).timeout - queue_free() still waits for something before it runs, it's just waiting on the flash timer now instead of the sound. $Hit.play() still runs

immediately, so the hit sound still plays; just keep sfx_hit.wav short (around a tenth of a second or less), since a longer custom hit sound is no longer explicitly waited on and could get cut off.

10. Run the scene and play through a full round - each hit should flash the enemy white briefly, and a small burst should appear at that spot for a moment.

Highlighted lines are new this week; everything else is code you already have.

```
# Enemy's hit-detection function (revised):
const FLASH_TEXTURE = preload("res://enemy_invader_white.png")

func _on_hitbox_area_entered(area):
    area.get_parent().queue_free()
    $Hitbox.set_deferred("monitoring", false)
    $Hit.play()
    var explosion = explosion_scene.instantiate()
    explosion.global_position = global_position
    get_tree().current_scene.add_child(explosion)
    texture = FLASH_TEXTURE
    destroyed.emit()
    await get_tree().create_timer(0.1).timeout
    queue_free()

# explosion.tscn's own script, attached to its Sprite2D root (NEW):
extends Sprite2D

func _ready():
    $Timer.timeout.connect(_on_timer_timeout)

func _on_timer_timeout():
    queue_free()
```

Level 2 - Complete (Alpha/Beta)

The flash and explosion play correctly on every single hit, with no leftover explosions, no enemies stuck mid-flash, and no change to score/lives/wave timing.

- Play a full round: every single hit shows a flash and an explosion, with no enemy left stuck mid-flash.
- Confirm score, wave-clear, and win/lose all still happen at exactly the moments they did last week - destroyed.emit() runs before the await, so GameManager hears about the hit immediately; only queue_free() is delayed.
- Confirm explosion.tscn instances never pile up - each one removes itself via its own Timer, with no help from the Enemy.
- Confirm the hit sound still plays audibly on every hit - $Hit.play() is unchanged and still unawaited, it just isn't explicitly protected from being cut short anymore the way await $Hit.finished used to protect it.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: instead of one flat instant swap, alternate texture between FLASH_TEXTURE and the Enemy's normal texture 2-3 times across the 0.1 seconds (with tiny awaits between) for a flicker instead of a single snap.
- Visual: scale the explosion up slightly over its 0.2-second lifetime using a Tween, so it feels like it's expanding rather than just appearing and vanishing.

- Structure: notice explosion.tscn knows nothing about score, lives, or waves - it's a fully self-contained scene, the same discipline as this course's bullets and enemies.

If Something's Wrong

- Enemy vanishes instantly, no flash visible: confirm await get_tree().create_timer(0.1).timeout was actually added, and that queue_free() comes AFTER it (destroyed.emit() should stay BEFORE the await, not move with it).
- Enemy disappears but never visibly turns white: confirm enemy_invader_white.png was actually imported and FLASH_TEXTURE's preload path matches where it landed in the project - a missing/misnamed path fails silently and the sprite just keeps its normal texture.
- Explosion never appears, or an error about a missing scene: confirm explosion_scene is preloaded/exported and assigned to the actual explosion.tscn file.
- Explosions appear but never disappear, piling up on screen: two separate things can cause this, and it's worth checking both. First, confirm the Timer's Autostart is actually turned ON in the Inspector - Wait Time and One Shot alone don't start the countdown, and a Timer that never starts never fires timeout. Second, confirm explosion.tscn's own root node (the Sprite2D) actually has its own script attached, with a _ready() that connects $Timer.timeout to a function calling queue_free() - this is a NEW, separate script from enemy.gd; enemy.gd only ever spawns the explosion and has no way to reach into it and clean it up later. Also double-check the Timer is a DIRECT CHILD of the script's own node (not nested under some other wrapper node) - $Timer only finds a direct child, not a grandchild.
- Two hits on the same enemy cause an error: confirm area.get_parent().queue_free() (the bullet) and $Hitbox.set_deferred("monitoring", false) both happen immediately, before the await, so a second overlapping bullet can't re-trigger this same function on an enemy already mid-flash.
- Enemy never actually disappears, even though it flashes correctly: this almost always means area.queue_free() was called instead of area.get_parent().queue_free() somewhere - area IS the bullet's own Hitbox child node, not the Bullet scene's root, so destroying only the Hitbox leaves the actual Bullet (sprite, movement, everything) still alive on screen, just no longer able to detect collisions.

Want More?

Student Resource List > Godot section, or docs.godotengine.org